

# Cilium

## eBPF-Powered Networking, Observability, and Security for Kubernetes

---

A study report on cloud native network security and policy enforcement

*Network Security & Monitoring — Team 5*

September 16, 2026

Written as part of the open-source NSM tools assessment.

## Contents

---

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                | <b>2</b> |
| <b>2</b> | <b>How eBPF Powers Cilium</b>                      | <b>2</b> |
| 2.1      | What eBPF Is . . . . .                             | 2        |
| 2.2      | The eBPF Datapath . . . . .                        | 2        |
| <b>3</b> | <b>Network Policy and Security Enforcement</b>     | <b>3</b> |
| 3.1      | Kubernetes Network Policy . . . . .                | 3        |
| 3.2      | CiliumNetworkPolicy: Layer 7 Enforcement . . . . . | 4        |
| 3.3      | Identity-Based Security Model . . . . .            | 4        |
| <b>4</b> | <b>Observability with Hubble</b>                   | <b>5</b> |
| 4.1      | What Hubble Is . . . . .                           | 5        |
| 4.2      | Flow Logs and DNS Visibility . . . . .             | 5        |
| 4.3      | Hubble in an Observability Stack . . . . .         | 5        |
| <b>5</b> | <b>Cilium in the Cloud Native Security Stack</b>   | <b>6</b> |
| <b>6</b> | <b>Limitations</b>                                 | <b>6</b> |
| <b>7</b> | <b>Conclusion</b>                                  | <b>7</b> |

## 1. Introduction

---

Cilium is an open-source networking and security project that uses eBPF — Extended Berkeley Packet Filter — to secure and monitor container traffic [1]. It was originally developed by Isovalent and became a graduated CNCF project in October 2023 [2].

Cilium works as a Container Network Interface (CNI) plugin for Kubernetes. Most CNI plugins only handle basic IP routing, but Cilium does much more: it routes traffic between pods, enforces network policies, handles load balancing, and records detailed flow data — all directly inside the Linux kernel.

What makes Cilium different is eBPF. Instead of using iptables rules, Cilium compiles its policy logic into small programs that run inside the kernel. This makes enforcement fast and does not require a separate user-space process [3].

Cilium is not a detection tool that generates alerts. It is the network layer itself. When a packet violates a policy, Cilium drops it before it ever reaches the destination pod — enforcement happens at the kernel level, before the application is involved.

## 2. How eBPF Powers Cilium

---

### 2.1. What eBPF Is

eBPF is a Linux kernel technology that lets small programs run inside the kernel in response to events such as network packets or system calls, without changing kernel source code or loading kernel modules [3]. Before a program is loaded, the kernel verifies it to make sure it cannot crash the system or access memory it should not. This makes eBPF safer than kernel modules and faster than running logic in user space.

For networking, eBPF programs attach at the TC (Traffic Control) hook inside the kernel, intercepting packets as they arrive at or leave a network interface. This happens earlier in the packet path than iptables, so policy decisions are made with less overhead [4].

#### Key insight

Cilium fully replaces iptables on nodes where it runs. With iptables, rules are checked one by one and the list grows with every service and policy added — this becomes slow in large clusters. Cilium's eBPF datapath uses hash-table lookups that stay fast regardless of cluster size [5].

### 2.2. The eBPF Datapath

When Cilium is installed on a Kubernetes node, it attaches eBPF programs to the network interface of every pod on that node. These programs handle three things:

- **Packet forwarding:** Cilium stores routing information for all pods and services in eBPF maps — fast in-kernel lookup tables. Traffic between pods on the same node is forwarded directly in the kernel without going through the normal network stack.
- **Policy enforcement:** Each pod's virtual interface gets an eBPF program that checks incoming packets against the active network policy and drops them if they are not allowed.
- **Load balancing:** Cilium handles Kubernetes service traffic in eBPF, replacing kube-proxy. The virtual IP of a service is translated to a real pod IP inside the kernel, removing the need for a separate user-space proxy.

### 3. Network Policy and Security Enforcement

---

#### 3.1. Kubernetes Network Policy

Cilium supports the standard Kubernetes `NetworkPolicy` API [6]. A `NetworkPolicy` uses label selectors to choose which pods it applies to, and defines which ingress and egress traffic is allowed. Any traffic that is not explicitly permitted is blocked by default. The example below allows only traffic from pods labelled `frontend` to pods labelled `backend` on port 8080:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: production
spec:
  podSelector:
    matchLabels:
      tier: backend
  ingress:
    - from:
        - podSelector:
            matchLabels:
              tier: frontend
      ports:
        - protocol: TCP
          port: 8080
```

### 3.2. CiliumNetworkPolicy: Layer 7 Enforcement

Standard Kubernetes NetworkPolicy only works at Layer 3/4 — it can allow or block traffic based on IP addresses and ports, but it cannot look inside the connection. Cilium extends this with its own CiliumNetworkPolicy resource, which supports Layer 7 rules for HTTP, gRPC, Kafka, and DNS [7].

The example below allows only HTTP GET requests to a specific path, while blocking everything else on the same port:

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: allow-get-only
spec:
  endpointSelector:
    matchLabels:
      app: api-server
  ingress:
    - fromEndpoints:
        - matchLabels:
            app: frontend
      toPorts:
        - ports:
            - port: "80"
              protocol: TCP
          rules:
            http:
              - method: GET
                path: "/api/v1/.*"
```

This is a useful security boundary. If the frontend pod is compromised and an attacker tries to send a POST request to /admin, Cilium drops it at the kernel level before it reaches the API server.

### 3.3. Identity-Based Security Model

Traditional network security uses IP addresses to decide who can talk to whom. In Kubernetes this does not work well, because pod IPs change every time a pod is restarted or rescheduled.

Cilium solves this with an **identity-based security model** [8]. Each pod gets a numeric identity based on its Kubernetes labels. This identity travels with the packet and is used

for policy decisions instead of the IP address. As a result, policies stay correct even as pods come and go, without any manual updates.

## 4. Observability with Hubble

---

### 4.1. What Hubble Is

Hubble is Cilium's built-in observability tool [9]. It taps into the same eBPF programs that enforce policy to capture a live feed of all network flows across the cluster. Because it runs alongside the datapath inside the kernel, it does not need a separate agent or network tap.

Hubble exposes its data through three interfaces: a CLI (`hubble observe`), a gRPC API, and a web UI. It can answer questions like: which pods talked to each other in the last few minutes, which connections were dropped by policy, and which DNS names a pod looked up.

### 4.2. Flow Logs and DNS Visibility

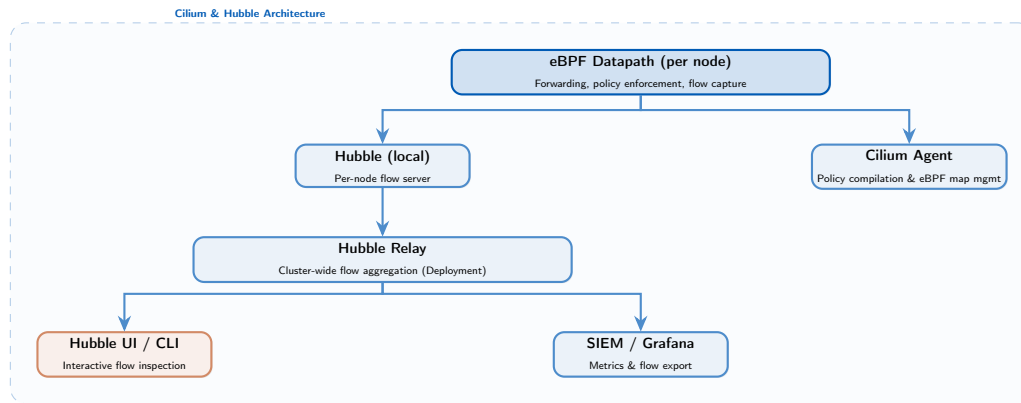
Every flow recorded by Hubble includes the source and destination pod name, namespace, labels, IP, port, protocol, and whether the connection was allowed or dropped. For HTTP traffic, Hubble also logs the method, URL, and response code.

DNS visibility is one of Hubble's most useful features. It records every DNS query and response from every pod in the cluster, giving a complete audit trail without any changes to the applications. This makes it easy to spot pods that resolve domains they should not, or detect DNS-based data exfiltration.

### 4.3. Hubble in an Observability Stack

Hubble exports data in two formats that work with common observability tools:

- **Prometheus metrics:** Cilium and Hubble expose metrics for packet drops, policy verdicts, DNS query counts, and HTTP error rates. These can be visualised directly in Grafana [10].
- **Flow export:** Hubble Relay collects flows from all nodes and can send them to a SIEM or stream processor via the Hubble flow API or an OpenTelemetry exporter.



## 5. Cilium in the Cloud Native Security Stack

Cilium sits at the **network enforcement layer** of a cloud native security setup. It does not generate alerts about suspicious behaviour — that is the job of tools like Suricata or Falco. Instead, Cilium ensures that pods can only communicate with what they are explicitly allowed to reach, and only on the expected protocols and paths.

This limits the damage when something goes wrong. If a pod is compromised, Cilium prevents it from reaching other internal services or making unexpected outbound connections. Other detection tools then work in an environment where the attack surface is already reduced.

Cilium also works with service mesh setups. When used together with Envoy — which Cilium can embed as a transparent L7 proxy — it provides mutual TLS between pods, traffic shaping, and request-level observability, all without changing application code [11].

## 6. Limitations

Cilium is a capable platform, but it has some real constraints worth knowing:

- **Kernel version requirements:** Cilium needs a fairly modern Linux kernel. The minimum for production is 4.19, but features like the BPF-based host firewall and WireGuard encryption require kernel 5.10 or later [12]. Most managed Kubernetes services (EKS, GKE, AKS) meet this, but older bare-metal setups may not.
- **Operational complexity:** Cilium replaces kube-proxy and, depending on the setup, the underlying CNI as well. When something goes wrong with networking, debugging requires knowledge of eBPF concepts and Cilium-specific tools like `cilium monitor` and `hubble observe`. This is harder than troubleshooting simpler CNI plugins like Flannel or Calico.
- **No syscall visibility:** Cilium only sees network traffic. It cannot detect a process reading a sensitive file, spawning unexpected child processes, or escalating privileges.

A separate host-level agent is needed for that.

- **L7 policy adds latency:** Enforcing HTTP or gRPC policies requires routing traffic through an embedded Envoy proxy. This adds latency compared to L3/L4 enforcement, which stays entirely in-kernel. L7 policies should be applied only where needed.
- **Policy management at scale:** Maintaining CiliumNetworkPolicies across a large cluster with many services can get complicated. Without good tooling it is easy to end up with policies that are either too permissive or incomplete.

## 7. Conclusion

---

Cilium fills a gap that packet-capture tools and syscall monitors leave open: network enforcement at the kernel level. By using eBPF, it provides fine-grained network segmentation for Kubernetes workloads with performance that iptables-based solutions cannot match.

Its identity-based model keeps policies accurate across the dynamic lifecycle of pods, and Layer 7 support lets teams set security rules at the HTTP or gRPC level instead of just TCP/IP. Hubble sits on top of the same datapath and gives cluster-wide flow visibility and DNS logging without extra instrumentation.

The main trade-off is complexity. Cilium takes more effort to understand and operate than a basic CNI plugin. But for teams running Kubernetes at scale, the combination of fast networking, strong policy enforcement, and built-in observability makes it the most capable open-source CNI available today.

## References

---

- [1] Isovalent / Cilium Authors, *Cilium Documentation*, <https://docs.cilium.io/en/stable/>, 2024.
- [2] Cloud Native Computing Foundation, *Cilium CNCF Project Page*, <https://www.cncf.io/projects/cilium/>, 2024.
- [3] eBPF Community, *What is eBPF?*, <https://ebpf.io/what-is-ebpf/>, 2024.
- [4] Isovalent, *eBPF Datapath — Cilium Documentation*, <https://docs.cilium.io/en/stable/network/ebpf/>, 2024.
- [5] T. Graf, *Cilium: Replacing iptables with eBPF*, KubeCon Talk / Isovalent Blog, <https://isovalent.com/blog/post/2021-12-08-cilium-scalability/>, 2021.
- [6] Kubernetes Authors, *Network Policies — Kubernetes Documentation*, <https://kubernetes.io/docs/concepts/services-networking/network-policies/>, 2024.
- [7] Cilium Authors, *Layer 7 Policy — Cilium Documentation*, <https://docs.cilium.io/en/stable/policy/language/#layer-7-examples>, 2024.
- [8] Cilium Authors, *Identity-Based Security — Cilium Documentation*, <https://docs.cilium.io/en/stable/concepts/security/identity/>, 2024.
- [9] Cilium Authors, *Hubble — Network Observability for Kubernetes*, <https://docs.cilium.io/en/stable/observability/hubble/>, 2024.
- [10] Cilium Authors, *Metrics — Cilium Documentation*, <https://docs.cilium.io/en/stable/observability/metrics/>, 2024.
- [11] Cilium Authors, *Cilium Service Mesh — Cilium Documentation*, <https://docs.cilium.io/en/stable/network/servicemesh/>, 2024.
- [12] Cilium Authors, *System Requirements — Cilium Documentation*, [https://docs.cilium.io/en/stable/operations/system\\_requirements/](https://docs.cilium.io/en/stable/operations/system_requirements/), 2024.